

COURSE REVIEW NOTES

Managing GKE at scale

A plain-language synthesis of the arguments in each module.
Follows the original lesson order.

Thirteen lessons · fleets, networking, mesh, security, identity, CI/CD, and AI

Google Cloud Architect · GKE Enterprise path · Managing GKE at scale

Written for review. Arguments only. Original slide decks remain the source of record.

How to use this document

This is a study aid, not a transcript. It restates the *arguments* of the Managing GKE at Scale course in everyday language so you can review the logic without flipping through every slide. Command snippets and YAML examples from the decks are left out on purpose. When you need the exact how-to, go back to the original lesson.

The source files are numbered as module 15, lessons 2 through 14. There is no lesson 1 in the folder. The course itself numbers sections 01–13. This document uses that course order.

What “argument” means here

Each module is treated as a claim: a problem exists, a GKE feature answers it, and there are trade-offs. The review below is organized that way, so you can ask “why would I use this?” instead of memorizing product names.

Contents

- 01 Introduction to GKE at scale — why fleets exist
- 02 Multi-cluster architecture — how the platform is put together
- 03 Fleets and teams — grouping clusters and giving people a slice
- 04 Configuration at scale — GitOps, policy, and blueprints
- 05 Fleet networking — how clusters talk to each other
- 06 Cloud Service Mesh — traffic, security, and visibility between services
- 07 Mesh routing — where traffic goes and how it behaves
- 08 Service mesh security — encryption, identity, and who may call whom
- 09 Multi-cluster mesh networking — one network vs many networks
- 10 Identity and authentication — people and workloads logging in
- 11 Security posture — seeing and preventing problems
- 12 CI/CD at scale — building, shipping, and locking the supply chain
- 13 GKE and AI — training, serving, and paying for accelerators

01 Introduction to GKE at scale

Lesson 2 · Why fleets exist

Kubernetes is the usual way to run containers. The course’s opening claim is that **the hard part is not Kubernetes itself — it is running many clusters, for many teams, in more than one place, without inventing a custom control plane for every environment.**

The problem

Large companies rarely live in one cloud. A typical enterprise already uses more than two providers. The reasons are practical, not fashionable:

- **Rules.** Public-sector or regulated customers may require data to stay in a country or on a specific kind of infrastructure.
- **Bargaining power.** Using more than one vendor reduces lock-in and helps in price talks.

- **Resilience.** If one provider has a long outage, a second home for the workload can keep the business running.
- **History.** Mergers and old contracts leave you with clouds you did not choose.

Once you have many clusters — Google Cloud, on-premises, AWS, Azure, even edge sites — the day-to-day work explodes: identity, policy, upgrades, logging, and traffic all have to be repeated. Homegrown tooling is the usual trap. GKE's answer is to treat those clusters as one **fleet**.

What a fleet is

A fleet is a logical group of Kubernetes clusters that you manage together. It can include clusters on Google Cloud and elsewhere. This course focuses on GKE on Google Cloud, but the same ideas apply when clusters sit outside Google. Each fleet lives in a **fleet host project**: the project you use to look at and steer the group, even if some clusters belong to other projects.

GKE is presented as more than “managed Kubernetes.” Around the clusters sit configuration and policy tools, team access, a service mesh, identity, and observability. The pitch is a single operating model: create, update, and govern clusters wherever they run, and modernize existing apps without rewriting them from scratch.

Sameness and trust

Two ideas make a fleet more than a folder of clusters. **Sameness** means objects with the same name — especially namespaces — are treated as the same thing across clusters. If namespace *foo* exists in three clusters, a security policy aimed at *foo* can apply in all three. You do not re-invent the policy per cluster.

Trust means those clusters and teams are allowed to share a security and identity boundary. When they trust each other, you can manage resources at fleet level. If they do not, you split them. Trust is the price of simpler operations.

How to group things

Align fleets with how the company already thinks: production vs test vs development, or different departments. Put clusters that talk to each other often in the same fleet. Aim for the **largest fleet that still preserves sameness and trust**. Smaller slices come later, through team scopes, not by multiplying fleets without reason.

Teams get a slice of the fleet through **scopes**. Two teams can share a cluster and still keep their own namespaces. Teams sometimes break perfect sameness — different app needs, test vs prod habits, org silos, compliance, legacy systems, or simply resistance to change. That tension is expected. The course returns to teams in a later module.

The supporting toolkit (preview)

The introduction already names the pieces the rest of the course unpacks: Connect Agent (a small workload that opens a secure tunnel from a cluster back to Google, even without a public IP), load balancing including multi-cluster ingress, Google or third-party login (SAML/LDAP where needed), Policy Controller, Binary Authorization, network policy, Cloud Service Mesh, and dashboards plus Cloud Logging/Monitoring (or Prometheus, Grafana, Elastic, Splunk).

***In one sentence.** Run many clusters as one trusted group with shared names and shared rules, instead of operating each cluster as a snowflake.*

02 Multi-cluster GKE architecture

Lesson 3 · How the platform is put together

This module's argument: **centralize the operating system of your clusters in GKE, then register them to a fleet so mesh, policy, and config tools can attach.** Think of GKE as the OS and each cluster as hardware it drives the same way, even if the hardware lives in another cloud.

Quick refresh

Containers package an app with what it needs to run. Kubernetes places those containers, finds them, balances load, rolls versions forward and back, watches health, and heals failures. GKE is the managed version: Google handles provisioning, networking, load balancing, security, and upgrades.

A single environment, then many

A typical environment (say production) is not one cluster. It is several clusters in more than one region, a fleet config layer, Cloud Load Balancing in front, plus Config Sync, Policy Controller, a workload identity pool, and a service mesh. Logging and monitoring sit beside all of that. Non-prod and dev look the same, just smaller.

As microservices multiply, Cloud Service Mesh becomes the dedicated layer that steers and secures traffic between services. Tenants are modeled as **namespaces**: quotas, isolation, and permissions live there. Each service should have its own build-and-deploy pipeline, reused from development through production, so the path does not change with the environment.

Who owns what

Google's reference layout splits Google Cloud projects by job. A common project holds Cloud Build, Artifact Registry, and Cloud Deploy. Separate projects hold prod, non-prod, and development clusters (often two clusters in prod for availability, one in dev). Application teams deploy into namespaces. They do not resize clusters or create new ones. Platform teams own the infrastructure. That split of duty is the architecture, not an afterthought.

Turning fleets on

You can enable fleets in the console, with gcloud, or with Terraform. New fleets can auto-register clusters. Managing clusters on other clouds needs extra APIs (multi-cloud, connect, connect gateway, resource manager, logging, monitoring). Two location decisions matter: where the clusters physically run, and which Google Cloud region stores fleet control data (keys, resource info) for residency rules. Google maps its regions to AWS and Azure regions so you can keep management close to the data.

Connecting and logging in

Connect Agent is required for clusters outside Google Cloud. It handles API calls over TLS, works through NAT and firewalls, and does not need a public IP. After registration, Connect Gateway is the front door: from a laptop, a CI pipeline, or the console, you reach member clusters the same way. Login can be Google identity, a third-party IdP (OIDC, Entra ID, Okta), or bearer tokens.

***In one sentence.** Build environments as copies of the same pattern, separate platform from app teams, and register every cluster so one control plane can see them all.*

03 Fleets and teams

Lesson 4 · Grouping clusters and giving people a slice

One cluster is enough when you are small. Multiple clusters help you split workloads and keep prod away from development. The argument here is that **fleets are the alternative to writing your own multi-cluster tooling**, and that teams are how you share a fleet without giving everyone the keys.

What fleets buy you

- Same namespaces, services, workload identities, and mesh identity across member clusters.
- Each fleet-aware resource belongs to only one fleet — one source of truth, fewer clashes.
- A trust domain: a policy set on the fleet reaches every cluster in it.
- You keep the benefits of many clusters (isolation, blast-radius limits) while governing from the center.

Fleet-aware extras include workload identity pools, multi-cluster gateways, Cloud Service Mesh, and Git-based config and policy. Register new Autopilot or Standard clusters at create time, or attach existing ones, including to a fleet in another project.

Isolation versus consistency

The course walks a four-cluster example (two prod regions, staging, on-prem dev) with frontend and backend services. The design choice is always the same: more fleets give isolation; fewer fleets give sameness.

- **One fleet for everything.** Namespaces are renamed to show the environment (frontend-prod, frontend-staging). Easy consistency. Isolation then depends on mesh authorization, which is easy to misconfigure.
- **Prod fleet and pre-prod fleet.** Production is walled off. Developers can still point a local frontend at a staged backend. Environments are no longer identical — that is the cost.

If two sides cannot trust each other, you can join two meshes, but you pay in extra operations. Prefer trust inside one fleet when you can.

Scopes and fleet namespaces

A **team scope** is the admin's unit: which clusters a team may use, plus logs, metrics, errors, and rollout order for that team. Members log in with Google groups through Connect Gateway. A **namespace** is the developer's unit: deployments, services, pods, isolation, and Kubernetes RBAC via kubectl.

Example: backend team on clusters 1 and 2 in namespace *backend*; frontend team on clusters 2 and 3 in two frontend namespaces. Cluster 2 is shared; namespaces are not. Leads get admin in the scope; others get edit. Setup is: enable APIs, prefer Google Groups, create the scope, bind clusters, create fleet namespaces, grant Scope Admin / Editor / Viewer. The Teams page shows CPU, memory, disk, errors, restarts, and a cost estimate.

Upgrades without a big-bang

The fleet host project is the control room. **Rollout sequencing** upgrades clusters in waves (test, then staging, then prod). Control planes go first, then nodes, still respecting maintenance windows. **Soak time** is the wait between waves so you can watch for damage. You can override soak time, skip a cluster with a maintenance exclusion, or pull a cluster out if its version does not match the target. Statuses include ineligible, pending, in progress, soaking, forced soaking (after 30 days), and complete. Team-based sequences need each team on its own clusters in separate fleets.

***In one sentence.** Group clusters where trust and names can be shared; give teams scopes, not whole fleets; upgrade in waves with time to soak.*

04 Managing configuration at scale

Lesson 5 · GitOps, policy, and blueprints

The argument: **when many teams change many clusters by hand, the live system stops matching anyone's idea of the truth.** That gap is called drift. GitOps is the fix: the desired state lives in Git; clusters pull it; humans stop kubectl-patching production as the main workflow.

How complexity grows

A small team can own one cluster. Then you need staging, test, and prod. Then a second team appears. Then you must stay compliant in more than one region. Every extra tenant, namespace, and cluster multiplies the policies you must apply. Meanwhile Kubernetes is declarative on paper but people apply changes imperatively. Team Blue edits a cluster and never writes it down. Team Green overwrites Blue. Nobody can say what “correct” is.

The three tools

Platform admins commit configs and policies to Git (cloud or on-prem). Then:

- **Config Sync** copies that Git state onto clusters, continuously. It is auditable (every change is a commit), reversible, transactional (group changes), and self-healing (it puts back what someone deleted).
- **Policy Controller** (Open Policy Agent Gatekeeper) is the guardrail. It checks every API request against rules you choose. Built-in library plus custom constraints. You do not need to write Rego to use a template — you fill YAML parameters.
- **Config Controller** extends the same idea to Google Cloud resources (Cloud SQL, Pub/Sub, IAM). Cloud looks like Kubernetes objects, so GitOps covers more than pods.

Olivia, the platform admin in the story, stores a Config Management file that points clusters at a Git branch and folder. Config Sync pods keep desired and observed state aligned. When a teammate deletes a namespace, Sync recreates it within seconds. RootSync is cluster-wide; RepoSync is namespace-scoped. A dashboard shows sync status, errors, and drift.

Shift-left policy

Policy lives in the Kubernetes API, so kubectl, Terraform, and CI pipelines all hit the same rules. Benefits claimed: consistent enforcement, early detection, one place to edit, better posture. Typical rules: pods must not run as root; deployments must set CPU/memory limits; network policies must exist; images must come from trusted registries; labels for chargeback; PCI/HIPAA-aligned constraints. A constraint names the template, what it matches, and the parameters (for example: every namespace needs a *team* label).

Blueprints and landing zones

The Config Management suite is a lot. **Blueprints** are packaged, versioned starting points — Kubernetes and Terraform — so you do not design a secure cluster from a blank page. A **landing zone** is the “where”: org structure, who may do what, billing, logging, networking. An **enterprise application blueprint** is the “what and how”: cluster shape, microservices, CI/CD, app security, monitoring, developer workflow. Admins pick a blueprint, customize it (often with kpt), store it in Git, let Cloud Build validate, and let Config Sync apply it.

In one sentence. Put the desired state in Git, sync it, refuse illegal states at the API, and reuse blueprints so every environment is born compliant.

05 Fleet networking

Lesson 6 · How clusters talk to each other

Apps now span regions, zones, and sometimes clouds. The claim: **fleet networking is the fabric that makes those clusters behave like one network, instead of a pile of VPNs and one-off load balancers.**

Two directions of traffic

- **North–south** is in and out: internet or internal clients hit a load balancer, then a cluster.
- **East–west** is cluster to cluster: needed when services depend on each other, for failover, and for blue-green. Cloud Service Mesh is the main tool here.

Why put clusters in several places? Closer to users (latency), extra copies (availability), same country as the customer (compliance), compute next to data (cost and speed). Several clusters also isolate environments, limit blast radius, let you upgrade one side while the other serves traffic, and keep sensitive data off app clusters.

On Google’s network, a global HTTP(S) load balancer uses an anycast IP, so the same address is announced in many regions and traffic enters at the nearest Google point of presence. Premium Network keeps the rest of the path on Google’s backbone. Network Endpoint Groups (NEGs) point at Cloud Run, VMs, or pod IPs (container-native load balancing).

Finding pods, then finding them across clusters

Pods come and go, so you never hard-code their IPs. Inside one cluster, a Service object, Endpoints, kube-proxy, and DNS (CoreDNS) turn a name into a virtual IP, then into a live pod. That only works inside one cluster. Across clusters you need extra machinery.

Multi-cluster Services (MCS) export a service so other clusters can find it via Cloud DNS and a virtual IP. MCS is for GKE clusters with VPC-native networking. It is separate from Istio. It is a blueprint: it picks pods by label in member clusters, creates a derived (headless) Service, and that Service owns a NEG of pod IPs. Empty if no matching pods. You can limit which clusters participate — useful to protect the config cluster, do blue-green, or send path-based traffic to backends that only exist in some regions.

Multi-cluster Gateway

Multi-cluster Gateway (MCG) is the successor to Multi-Cluster Ingress. It follows the Kubernetes Gateway API (role-oriented: platform provides GatewayClass, operators create Gateways, app owners write HTTPRoutes). It provisions Google HTTP(S) load balancers and can split, mirror, and fail over on health. Class names ending in *-mc* mean multi-cluster. HTTPRoutes point at ServiceImports, not local Services, because the backend may live elsewhere. Weights on backends enable canary, blue-green, and A/B tests. GCPBackendPolicy adds connection draining, Identity-Aware Proxy, Cloud Armor, and request logging.

***In one sentence.** Use Google’s global load balancing for users at the edge, Multi-cluster Services to find pods in other clusters, and a multi-cluster Gateway to steer HTTP traffic with weights and failover.*

06 Cloud Service Mesh

Lesson 7 · Traffic, security, and visibility between services

Microservices used to bury retries, encryption, and metrics in application code. The argument: **put that cross-cutting work in a mesh beside the app, so developers write business logic and operators change network behavior without a redeploy.**

Two planes

The **data plane** is a sidecar proxy (usually Envoy) next to each service — or, for some gRPC apps, no sidecar at all (“proxyless”), talking to the control plane with the same xDS API. The **control plane** is the brain: routing rules, security policy, observability config. Istio is the open-source mesh. Cloud Service Mesh (CSM) is Google’s managed Istio: Google upgrades and scales the control plane, data plane, and related bits.

What the sidecar buys you

- Cleaner app code and faster changes to retries or auth without rebuilding services.
- One place to apply policy — closer to zero trust, with mTLS between services.
- Traffic tricks: canary, A/B, blue-green, circuit breaking, timeouts, fault injection.
- Metrics, traces, and logs for every hop (Cloud Monitoring, Trace, Logging).

Turn the mesh API on, optionally make managed CSM the default for new fleet members (Mesh CA only, in that mode). Apps join when you inject sidecars (label a namespace) or run proxyless gRPC. Injection at namespace scale is the production pattern. Traffic interception is done with init containers and iptables (needs NET_ADMIN/NET_RAW) or with the Istio CNI plugin, which avoids those extra permissions — often better for compliance.

A request’s path

Pod starts, sidecar is injected, iptables send all traffic to Envoy. Envoy learns routes from the control plane. Mesh CA hands out TLS certs. Client Envoy intercepts the call, may change protocol, opens mTLS. Server Envoy checks certs and filters (authz, quotas, telemetry), then the app. The response walks back the same way. Managed CSM does not let you load arbitrary Wasm filters into Google-managed Envoys — stability over DIY extensions.

Dashboards and limits

Mesh dashboards cover the golden signals (latency, traffic, errors; saturation is inferred from CPU, memory, and those signals). You set SLIs, SLOs, and error budgets, draw topology, and drill into QPS. CSM can be part of GKE or standalone (priced by clusters and clients; Mesh CA has no per-certificate fee). Compared with raw Istio, managed CSM does not give you a custom CA, arbitrary Envoy filters, extra telemetry backends, multi-network/IPv6 the same way, or Istio ambient mode.

***In one sentence.** Let Google run Istio for you: sidecars (or proxyless gRPC) take networking out of the app, at the cost of some Istio flexibility.*

07 Cloud Service Mesh routing

Lesson 8 · Where traffic goes and how it behaves

You configure the mesh with Kubernetes custom resources. The control plane reads them and pushes them to Envoys over xDS. The mnemonic: **VirtualService is where the request should go; DestinationRule is how to talk to that destination.**

The five Istio resources (plus two for VMs)

- **VirtualService** — routing rules on host, headers, path, port, source labels. Rules are checked top to bottom. Prefer fully qualified names in production; wildcards group similar hosts.
- **DestinationRule** — subsets (usually by version label), load balancing (round-robin, random, least-request, pass-through), session affinity, TLS mode, circuit breakers.

- **Gateway** — Envoys on the edge, not sidecars. Ingress in, egress out, east-west between clusters. Layer 4–6 (ports, TLS); VirtualServices bound to the Gateway do Layer 7.
- **ServiceEntry** — register something outside the mesh (public API, legacy, a VM, a service in another cluster) so mesh features apply. Unknown hosts otherwise just pass through, with no mesh control.
- **Sidecar** — shrink what a proxy listens to and whom it may call, instead of “all ports, whole mesh.”
- **WorkloadEntry / WorkloadGroup** — bring VMs or bare metal in. WorkloadEntry is one instance (static IP or DNS); WorkloadGroup is the template (ports, service account, health probes), like a Deployment for non-Kubernetes.

Traffic splitting uses weights on subsets, independent of how many pods sit behind each version — canary is a percentage, not a replica count. Ingress flow: Google load balancer → NodePort → ingress-gateway pod → VirtualService → destination. Egress: ServiceEntry for the external host, Gateway for outbound, optional VirtualService. Useful when nodes have no public IP.

Resilience is configuration, not library code

- **Timeouts** — Envoy HTTP timeouts are off by default; set them or calls can hang.
- **Retries** — default is two retries; spacing is chosen by the mesh so you do not stampede the victim.
- **Circuit breakers** — cap connections; eject hosts that throw repeated 5xx (outlier detection).
- **Fault injection** — delay or abort a percentage of calls to see if callers cope. Delay and abort are independent.
- **Mirroring (shadowing)** — copy live traffic to a second version out of band. Good for a dark launch or for watching for odd paths and cleartext.

In one sentence. Route with VirtualService, shape the destination with DestinationRule, open the edge with Gateway, and test failure with timeouts, breakers, faults, and mirrors.

08 Service mesh security

Lesson 9 · Encryption, identity, and who may call whom

Many small services mean many network hops. The risks named are eavesdropping, impersonation, unconstrained access to data, and denial of service. The security goals are the usual four: **encrypt, authenticate, authorize, and (optionally) limit load with quotas**.

Encryption and mTLS, in plain words

If Service A sends cleartext, every router in the path can read or change it. TLS uses a public key to encrypt and a private key to decrypt. Microservices talk both ways, so they also share a symmetric session key. That is still not enough if a liar claims to be Service B. Certificates from a trusted authority prove identity. When both sides show certs, that is **mutual TLS (mTLS)**. Doing this by hand for hundreds of services is miserable. The mesh automates it.

Mesh CA issues and rotates X.509 certs. On pod start, the Istio agent asks for a cert, Mesh CA returns it, Envoy gets the material via SDS, and rotation repeats. **Secure naming** checks that the cert’s identity is allowed to run that service — so a hijacked DNS record pointing at a fake pod still fails.

Choosing the protocol

DestinationRules set TLS for a specific upstream (ISTIO_MUTUAL, custom MUTUAL, SIMPLE, or DISABLE). PeerAuthentication sets the default for a mesh, a namespace, or one workload. More specific wins; if two policies at the same level conflict, the oldest wins. Modes: STRICT (must be mTLS), PERMISSIVE (negotiate, good for

migration), DISABLE (testing only), UNSET (inherit, else permissive). Layer STRICT last, after everyone can speak mTLS.

People, not only services

End users prove themselves with JWTs from an OIDC provider (Auth0, Keycloak, Firebase, and so on). RequestAuthentication says where the token is, who issued it, and how to check the signature. No token can still pass this check — you must combine it with authorization if anonymous should be denied. Authservice can run the browser OIDC dance at ingress. For employees, **Identity-Aware Proxy** sits on the HTTPS load balancer: Google sign-in, then IAM (IAP-secured Web App User). Ingress terminates TLS and forwards to the mesh gateway. IAP can mint a RequestContextToken with a custom audience for finer mesh rules.

Authorization after authentication

Sidecars and edge proxies are policy enforcement points. An AuthorizationPolicy has a selector (which workloads), an action (ALLOW, DENY, AUDIT, CUSTOM), and rules: from (who), to (methods and paths — Layer 7, unlike Kubernetes NetworkPolicy), when (extra conditions such as JWT issuer). That is how you say “only this service account may GET /info on httpbin.”

***In one sentence.** Encrypt every hop with mesh-managed mTLS, prove users with JWT or IAP, then allow or deny with AuthorizationPolicy — not with hope and a flat network.*

09 Multi-cluster networking with Cloud Service Mesh

Lesson 10 · One network versus many networks

A mesh that spans clusters is not one recipe. **The shape of the network decides the work:** same VPC is mostly “turn it on”; different VPCs or other clouds need an east-west gateway and extra trust.

Same network (east–west inside a VPC)

Create and register clusters, install managed CSM with automatic management, optionally add ingress (north–south) and a multi-cluster Gateway, label namespaces for sidecar injection, deploy. Services in one cluster can call the other because each control plane can see the other’s endpoints.

- **Multi-region / different subnets:** write firewall rules that allow all node, pod, and service CIDRs both ways.
- **Shared VPC:** the host project owns the network and the firewalls. Do not register the clusters to a fleet in that host project.
- **Private clusters:** API servers are not on the public internet. Exchange secrets that hold each other’s private API IPs for endpoint discovery, allow the caller’s pod CIDR as an authorized network, and enable global control-plane access so each mesh control plane can reach the remote GKE API.

Different networks

When VPC peering or Interconnect is not possible (multi-cloud, some hybrid setups), an **east-west gateway** proxies cluster-to-cluster traffic. You expose only the services that should be visible. The gateway may have a public address, but callers still need a trusted mTLS cert and workload identity. Add firewalls in production anyway.

Attached clusters (EKS, AKS, OpenShift, KIND, K3s, and similar) are Kubernetes you register with GKE but Google does not manage. Support varies; open-source Istio is sometimes the practical choice. Trust across those clusters needs a shared root CA, not a separate Mesh CA per island.

***In one sentence.** Same VPC: firewalls and private-API plumbing. Different networks: east-west gateway plus a common root of trust.*

10 Identity and authentication

Lesson 11 · People and workloads logging in

Open-source Kubernetes still has passwords and service-account tokens. Those do not scale to many clusters and many people. The argument: **centralize who a human is (Google or your company IdP) and stop shipping long-lived keys for workloads.**

People

A Kubernetes service account (KSA) is the identity of a process in a pod. Bearer tokens are simple for scripts and CI, but they are powerful if leaked, they expire, and they are awkward to audit. Each cluster wants its own token — that is the scaling problem.

Google Identity + IAM is the default for GKE on Google Cloud. **Connect Gateway** is the single front door for fleet members: discover clusters, connect to the API server, authenticate (Google, workforce federation, or GKE Identity Service groups), and authorize the same way everywhere. Setup: enable Hub APIs, register clusters, grant *gkehub.viewer* so people can fetch kubeconfig, then Kubernetes RBAC inside each cluster. *get-credentials* plus *kubectl* is the daily path.

GKE Identity Service uses OIDC so Okta, Auth0, Entra ID, and the like can log people in. User hits the cluster, is sent to the IdP, comes back with an ID token, Identity Service swaps it for a short access token for the API server. Configure per cluster or once per fleet. Benefits: no extra Kubernetes passwords, MFA and password policy stay at the IdP, better audit logs. Map IdP groups to RBAC. Least privilege, regular access reviews, MFA. Debug via ClientConfig, component logs, and the public docs.

Workloads

Old pattern: download a Google service account JSON key, store it as a Kubernetes secret, hope nobody copies it. **Workload Identity** (GKE on Google Cloud) links a KSA to a Google service account in a workload identity pool. The cluster is an OIDC issuer. The pod presents a KSA token and receives a short Google access token. Grant the GSA only what that app needs.

Workload Identity Federation is the same idea for work that is not on GKE: AWS, Azure, on-prem, other Kubernetes. You trust an external issuer, map its subjects to Google identities, and exchange tokens. No downloaded keys. Both approaches share the same hygiene: least privilege, namespace grouping, audits, and automation (Terraform or Config Sync).

***In one sentence.** Humans enter through Connect Gateway and your existing IdP; workloads borrow Google identity through federation, never through a JSON key in a secret.*

11 Security posture, compliance, and preventative controls

Lesson 12 · Seeing and preventing problems

CymbalCorp in the story runs public websites and payment systems on the same platform. One policy for all of that will fail. The argument: **security posture is layered, shared with Google, and something you watch continuously — not a one-time hardening checklist.**

The layers

- **Shared responsibility.** Google hardens the infrastructure; you harden apps, identities, and data.
- **Visibility.** The Security Posture dashboard is the cockpit.
- **Control plane.** RBAC, network policy, keep versions current.
- **Nodes.** Container-Optimized OS, Shielded Nodes, automatic patching.
- **Images.** Scan, Binary Authorization, trusted registries.
- **Network.** Policies, firewalls, private clusters.
- **Compliance.** Keep proving you still match the rules you claimed.

Goals: smaller attack surface, least privilege, detect and respond, stay audit-ready. GKE already has the basics; the enterprise extras are deeper visibility, more automation, and better threat detection.

The dashboard

It pulls audit logs, image metadata, and scan results into one view, compares your config with Google's recommended practices (RBAC, network policy, Pod Security Standards), and offers concrete fixes, sometimes with commands. Four areas: overview (how bad, how many), workload configuration, OS/image vulnerabilities, and threats from Google's intelligence. Sandra's example: a pod allowed to run as root. The dashboard flags it, groups by cluster and severity, and she sets `runAsNonRoot: true`. Review it often, turn on auto-remediation where you trust it, and still use Security Command Center and Binary Authorization beside it.

Nodes and images

Shrink images (distroless/scratch). Use COS. Auto-provision nodes so patching and repairs are default. Shielded Nodes: Secure Boot, vTPM identity, runtime integrity — aimed at bootkits and rootkits. Binary Authorization is the gate: unsigned or disallowed images never become pods.

Scan in CI, not only in prod. Right after build, Artifact Analysis looks at OS packages; Advanced Vulnerability Insights adds language libraries (Go, Maven, JavaScript, Python). Policies can fail the build, write a report, or open a ticket. Security Command Center then rolls GKE findings into the rest of Google Cloud so you can automate response.

***In one sentence.** Watch posture in one dashboard, harden nodes and images by default, and fail the pipeline when a scan or signature check says no.*

12 CI/CD at scale in GKE

Lesson 13 · Building, shipping, and locking the supply chain

Modernizing apps is slow if every release is a heroics exercise. CI/CD is how you get the same outcome every time. DORA's research is the stick: elite teams ship much more often, with shorter lead time, fewer failures, and much faster recovery. The course's claim is that **Google's pipeline is modular on purpose** — swap pieces for tools you already have — and that scale, security, and integration are the three places pipelines usually break.

The toolchain

Cloud Code in the IDE; Secure Source Manager for Git; Cloud Build to compile and test; Artifact Registry to store and scan images; Cloud Deploy to promote releases; Logging and Monitoring once live. Runtimes include GKE, Cloud Run, Cloud Run functions, Firebase.

Cloud Build runs each step as a container on a shared `/workspace` volume. Steps can be sequential or parallel. Use Google’s builders (docker, go, kubectl, npm, ...) or community images. A YAML file lists the steps; triggers start on code change; the last step often creates a Cloud Deploy release.

Cloud Deploy is the delivery control plane: environments in order, optional human approval, easy rollback, audit trail. It uses Skaffold so Helm, Kustomize, or kpt can render manifests without baking that choice into the pipeline. A delivery pipeline YAML lists stages (dev → staging → prod). Targets can live in other projects. You can restrict rollouts to change-freeze windows, override with a special role, run verify tests and pre/post hooks, promote or abandon a release, and do canaries by splitting traffic with the mesh. IAM separates who may render, deploy, and approve.

Cloud Run vs managed Knative Serving

Both hide Kubernetes toil for HTTP services that scale with traffic. Cloud Run is fully managed on Google’s machines (pay per use, gVisor isolation, automatic URLs). Managed Knative Serving runs on *your* GKE (included in GKE, your machine types including GPUs, your cluster size as the scale limit, RBAC and mesh instead of only IAM). Use Cloud Deploy to roll new Knative revisions. Canary story: send 5–10% of traffic to the new revision, watch errors and business metrics, step to 25/50/100, or roll back. Same pattern can apply across hybrid clusters.

Private networks

Do not punch a hole in the firewall for Cloud Build. Use a **private worker pool** peered into your VPC: no public IP, VPC Service Controls, static ranges, more concurrency, regional (helps data residency). Cloud Deploy renders and deploys on Cloud Build, so the same pool can serve both. Peer your VPC to Google’s service producer network, create the pool, point builds and Deploy execution configs at it.

Lock the supply chain

Secure the source (review, SAST, known dependencies), the build (provenance, CVE scans, trusted bases), and the deploy (promotion, approvals, policy). Prefer known base images. Scan in Artifact Registry (regional repos, CMEK, VPC-SC, audit logs, repo-level IAM). Separate the person who says “this is ready” from the person who says “ship to prod.” Binary Authorization requires attestations (built here, scanned, QA’d) or an explicit allowlist for third-party images — outdated Nginx from Docker Hub is the classic failure. Config Sync then enforces what may land in the cluster at all.

***In one sentence.** Build in Cloud Build, promote with Cloud Deploy, keep workers inside your VPC, and refuse to run anything that is not signed, scanned, and allowed.*

13 GKE and AI

Lesson 14 · Training, serving, and paying for accelerators

Generative AI wants huge models, an open software stack, and many GPUs/TPUs. The closing argument: **GKE is a good home for that work because it already orchestrates mixed workloads, scales far, and speaks the same Kubernetes language your other apps use** — while Vertex AI remains the right managed choice when you do not need exotic control.

When to stay managed, when to use GKE

Managed (Vertex AI and friends): less ops, faster start, pay-as-you-go, Google-backed compliance. Unmanaged / GKE: you need custom training loops, odd dependencies, very low latency, special hardware layout, strict data residency or explainability, or you already run a hybrid estate. The AI Hypercomputer picture is: fast machines

and networks at the bottom, GKE or Compute Engine as orchestrator, JAX/PyTorch/Keras on top, and flexible buying (on-demand, spot, Dynamic Workload Scheduler).

Training

Training can run for days on big data. A GKE training platform: data in Cloud Storage, GPU (or TPU) node pools, TensorFlow or PyTorch jobs as pods, often driven by the Kubeflow Training Operator, optional Vertex AI Model Registry. Package code + framework + libraries in an image; use `tf.distribute` or `torch.distributed`; watch with TensorBoard. Kubeflow Pipelines stitch preprocess → train → evaluate; Cloud Build rebuilds the trainer. You scale the job, keep custom environments, and pick the hardware.

Serving

Serving wants low latency, high QPS, and safe rollouts (A/B a recommender, for example). Same cluster idea: load balancer, serving pods (TensorFlow Serving, TorchServe, KServe, or custom), models from Cloud Storage or Vertex. Horizontal Pod Autoscaling plus health checks. Prometheus/Grafana and Cloud Logging for latency, throughput, and errors. Extra serving benefits: performance and availability that you tune yourself.

Cost

Accelerators dominate the bill. Autopilot charges for what pods request, not for idle node leftovers — useful when jobs come and go. Standard mode is for people who want to pin every knob. Share GPUs (time-slicing, Multi-Instance GPU, NVIDIA MPS). Prefer TPUs on GKE when the model fits — often better price-performance. Shrink serving images (Triton, FasterTransformer), use smaller GPUs if latency allows, use Spot for interruptible training (large discounts), and mount model weights with Cloud Storage FUSE (cache, parallel downloads, no pre-copy into the pod). FUSE is credited with much faster training time and throughput on multi-epoch jobs.

In one sentence. Use GKE when AI needs custom scale and hardware; then spend your care on sharing accelerators, Autopilot vs Standard, and not copying giant models the slow way.

How the arguments connect

Read as one story: companies already run more than one cluster and often more than one cloud (01). GKE’s architecture is a copied environment pattern plus a fleet host (02). Fleets need sameness, trust, team scopes, and staged upgrades (03). Config and policy must come from Git, not from ssh (04). Users reach apps through global load balancing; services find each other with multi-cluster Services and gateways (05). Between services, a managed mesh takes over retries, encryption, and telemetry (06–08). When clusters do not share a VPC, you add east-west gateways and a shared CA (09). People log in through one gateway and an IdP; pods never hold Google keys (10). You watch posture and refuse bad images (11). You build and promote the same way every time, inside the private network, with signatures (12). AI is “just” another demanding workload on that same platform, with GPUs in the node pools (13).

A short checklist while you revise

Can I name the fleet and why those clusters share trust? Is desired state in Git? Do north–south and east–west have an explicit design? Is mTLS STRICT only where every client can speak it? Do humans use the IdP and workloads use Workload Identity? Does CI fail on critical CVEs, and does Binary Authorization block unsigned images? If GPUs are involved, is sharing or Spot or FUSE already in the cost plan?

This synthesis is for personal review of the course arguments. It is not official Google documentation and it omits commands, IAM role strings, and YAML that belong in the original lessons.